МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

Звіт

З ЛАБОРАТОРНОЇ №2

Варіант 12

на тему: **"Функції вищого порядку, lambda, композиція"**

Виконав:

Студент групи ФЕП-22

Сисак Ілля

Перевірив:

Аспірант

Остріков Данило Андрійович

Львів 2026

Мета. Ознайомитися з принципами побудови конвеєрів обробки даних у функціональному стилі на мові Python. Навчитися застосовувати функції вищого порядку (map, filter, reduce), анонімні функції (lambda), функціональну композицію (compose, pipe) та ледачі ітератори для перетворення колекцій записів без мутації вхідних даних на прикладі обробки даних мобільних абонентів (Варіант 12).

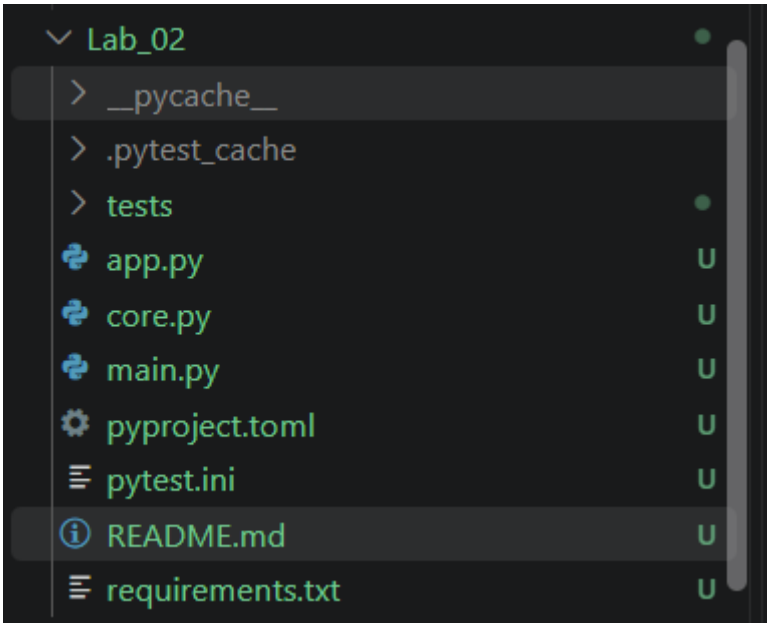
Теоретичні відомості

- **Чиста функція (Pure Function):** Функція, яка для однакових вхідних аргументів завжди повертає однаковий результат (детермінованість) і не викликає побічних ефектів (не змінює глобальний стан, не мутує вхідні аргументи, не виконує операцій введення/виведення).
- **Референтна прозорість (Referential Transparency):** Властивість виразу або виклику функції, за якої його можна замінити обчисленим значенням у будь-якому місці програми без зміни поведінки системи.
- **Побічні ефекти (Side Effects):** Взаємодія із зовнішнім середовищем або зміна стану поза межами локального контексту функції (виклики print(), читання/запис файлів, звернення до поточного часу time.time(), генерація випадкових чисел, мутування вхідних структур даних).
- **Ізоляція ефектів (Functional Core / Imperative Shell):** Архітектурний підхід, при якому вся обчислювальна та логічна обробка даних зосереджена в чистому ядрі (core.py), а всі побічні ефекти (введення, консольне виведення, робота з таймерами) винесені в зовнішню оболонку (app.py).
- **Параметризація та композиція через typing.Callable:** Передача алгоритмів та бізнес-правил (політик знижок, податків, критеріїв дозамовлення) у вигляді функцій-аргументів, що дає змогу будувати гнучкі та компоновані обчислювальні конвеєри за допомогою функцій вищого порядку (наприклад, compose(f, g) та make_multiplier(k)).

Хід роботи

Проєктування архітектури та типізації даних (Lab_02/):

- Організовано модульну структуру проєкту з розділенням на чисте ядро (core.py), CLI-оболонку (app.py), точку входу (main.py) та модуль тестів (tests/test_core.py).
- За допомогою TypedDict описано структуру запису абонента Record (поля id, name, tariff, payments, total), структуру підсумкової статистики StatsResult та тип акумулятора _Acc.



Реалізація чистого функціонального конвеєра (core.py):

- Реалізовано функції вищого порядку compose та pipe для гнучкого комбінування етапів обробки.
- Створено набір чистих трансформацій на основі map, filter та lambda, які створюють нові словники через розпакування ({**r, ...}) без мутації вхідних даних:
  - normalize_names() — приведення імені абонента до формату *Title Case* із видаленням зайвих пробілів;
  - with_total() — обчислення суми всіх платежів абонента (sum(payments));
  - boost_tariff(tariff, factor) — нарахування бонусного коефіцієнта для абонентів обраного тарифного плану;

- only_active(min_payment) — фільтрація активних абонентів за мінімальною сумою платежів;
- sort_by_total_desc() — сортування за спаданням підсумкової суми з додатковим ключем за зростанням id для вирішення нічиїх;
- take(n) — відбір перших N записів рейтингу за допомогою itertools.islice.

- Реалізовано функцію reduce_stats на базі functools.reduce, яка за один прохід обчислює три статистичні показники: кількість абонентів (count), загальну суму платежів (payments_sum) та середній платіж (avg_payment).
- Усі етапи об'єднано в єдиний конвеєр за допомогою функції build_pipeline.

```python
def build_pipeline(
    top_n: int = 3,
    tariff: str = "Premium",
    factor: float = 1.2,
    min_payment: float = 0.01,
) -> Callable[[Iterable[Record]], List[Record]]:
    return compose(
        take(top_n),
        sort_by_total_desc(),
        only_active(min_payment),
        boost_tariff(tariff, factor),
        with_total(),
        normalize_names(),
    )
```

```python
def reduce_stats(recs: Iterable[Record]) -> StatsResult:
    init: _Acc = {"count": 0, "payments_sum": 0.0}
    acc = reduce(
        lambda a, r: {
            "count": a["count"] + 1,
            "payments_sum": a["payments_sum"] + float(r.get("total", 0.0)),
        },
        recs,
        init,
    )
    avg = acc["payments_sum"] / acc["count"] if acc["count"] > 0 else 0.0
    return {
        "count": acc["count"],
        "payments_sum": acc["payments_sum"],
        "avg_payment": avg,
    }
```

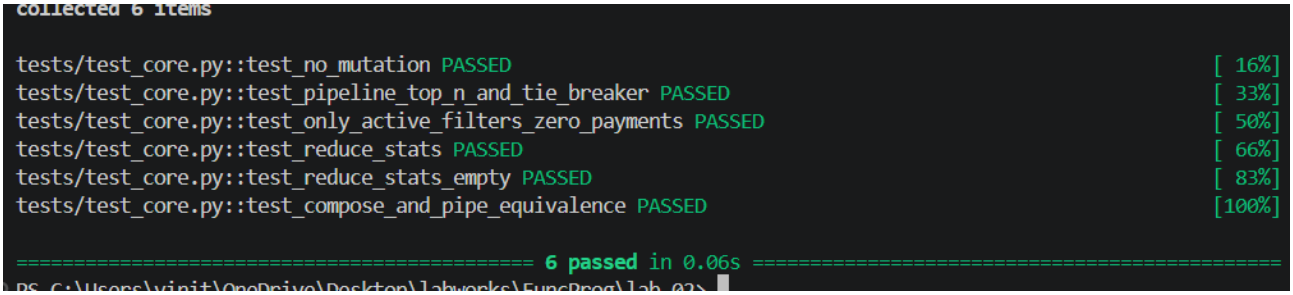
Розробка CLI-оболонки та точки входу (app.py, main.py):

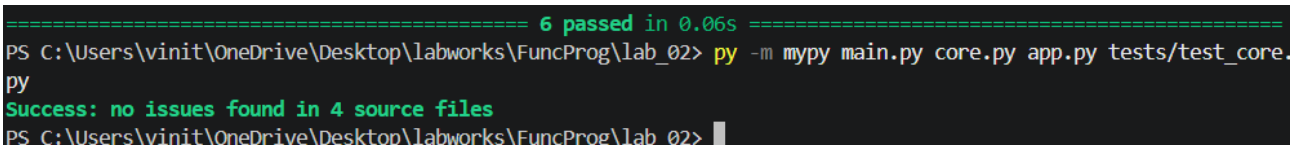
- У модулі app.py ізольовано операції введення/виведення: сформовано набір вхідних записів get_sample_data(), реалізовано розбір параметрів командного рядка через argparse (--top, --tariff, --factor) та форматований вивід таблиці рейтингу й статистики у render_report().
- У файлі main.py реалізовано стандартну точку входу main(), яка викликає run().

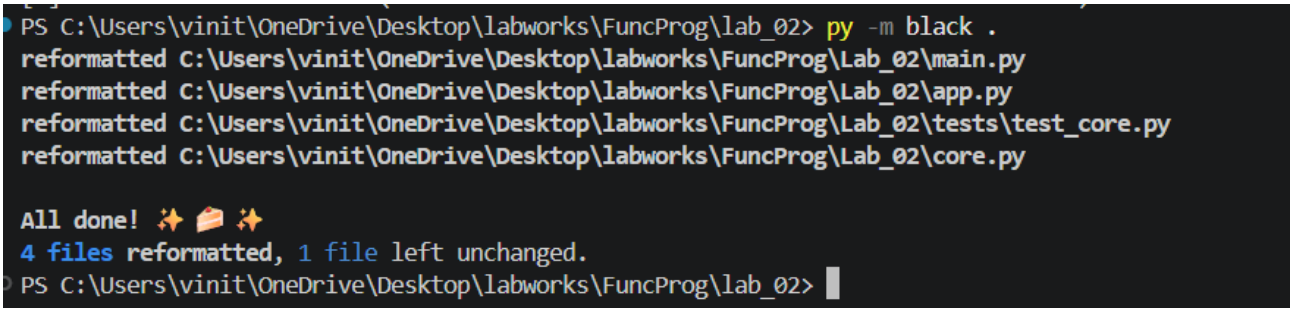
```
PS C:\Users\vinit\OneDrive\Desktop\labworks\FuncProg\lab_02> py main.py
============================================================
            РЕЙТИНГ МОБІЛЬНИХ АБОНЕНТІВ (TOP-N)
============================================================
 ID: 1   | Олена Коваль       | Тариф: Premium    | Сума:   540.00
 ID: 4   | Дмитро Бондар      | Тариф: Standard   | Сума:   540.00
------------------------------------------------------------
 Активних у рейтингу : 2
 Сума платежів       : 1080.00
 Середній платіж     : 540.00
============================================================
PS C:\Users\vinit\OneDrive\Desktop\labworks\FuncProg\lab_02> py app.py --top 2 --tariff Premium --factor 1.2
============================================================
            РЕЙТИНГ МОБІЛЬНИХ АБОНЕНТІВ (TOP-N)
============================================================
 ID: 1   | Олена Коваль       | Тариф: Premium    | Сума:   540.00
 ID: 4   | Дмитро Бондар      | Тариф: Standard   | Сума:   540.00
------------------------------------------------------------
 Активних у рейтингу : 2
 Сума платежів       : 1080.00
 Середній платіж     : 540.00
============================================================
```

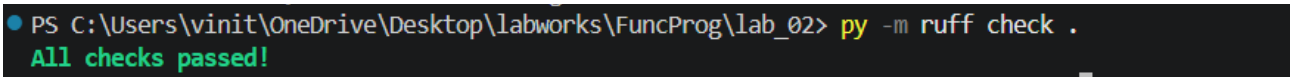
Автоматизоване тестування та статичний аналіз (tests/test_core.py):

- Написано набір unit-тестів для pytest:
  - test_no_mutation — перевірка незмінності вихідного списку записів після проходження конвеєра;
  - test_pipeline_top_n_and_tie_breaker — перевірка коректності бонусів, відбору Top-N та сортування при однакових сумах;
  - test_only_active_filters_zero_payments — перевірка відсіювання абонентів без платежів;
  - test_reduce_stats та test_reduce_stats_empty — перевірка розрахунку статистики для непорожнього та порожнього списків;
  - test_compose_and_pipe_equivalence — підтвердження ідентичності результатів конвеєрів, зібраних через compose та pipe.
- Проведено перевірку суворої типізації за допомогою mypy та форматування коду інструментами black і ruff.









Висновок. У ході виконання лабораторної роботи №2 було розроблено функціональний конвеєр для аналізу платежів та формування рейтингу мобільних абонентів (Варіант 12). Застосування функцій вищого порядку (map, filter, reduce), анонімних функцій (lambda) та функціональної композиції (compose, pipe) дозволило представити складну багатокрокову обробку даних у вигляді ланцюжка невеликих, незалежних і чистих трансформацій.

Усі перетворення виконуються без мутації вхідних структур даних, а побічні ефекти та взаємодія з користувачем через CLI повністю винесені з обчислювального ядра core.py в оболонку app.py. Коректність роботи конвеєра, відсутність мутацій, точність reduce-агрегації та сувора типізація підтверджені успішним проходженням тестів pytest і перевірок mypy, black та ruff.